

National University of Computer and Emerging Sciences



AL-2002: Artificial Intelligence Lab Manual 06

Department of Computer Science
FAST-NU, Lahore, Pakistan

[Table of Contents](#)

Objectives	3
1 Alpha–Beta Pruning for Minimax Algorithm.....	3
2 Hill Climbing Algorithm	4

Objectives

After performing this lab, students shall be able to understand implementation of the following :

- ✓ Game Playing using minimax algorithm with alpha beta pruning
- ✓ Hill Climbing Algorithm

1 Alpha–Beta Pruning for Minimax Algorithm

The basic Minimax algorithm explores all possible nodes in the game tree. For games with large branching factors, this becomes computationally expensive.

Alpha–Beta Pruning improves Minimax by:

- Avoiding exploration of branches that cannot affect the final decision
- Reducing the number of nodes evaluated
- Producing the same result as Minimax but faster

Alpha–Beta Pruning keeps track of two values during the search:

- α (alpha): The best value MAX can guarantee so far
- β (beta): The best value MIN can guarantee so far
- If at any point: $\alpha \geq \beta$ the remaining branches do not need to be explored because they cannot influence the final outcome.

Alpha (α) is used at MAX nodes. It is initially set to $-\infty$ and only updated when MAX finds a better value. Similarly Beta (β) is used at MIN nodes. It is initially set to $+\infty$ and updated only when MIN finds a better value

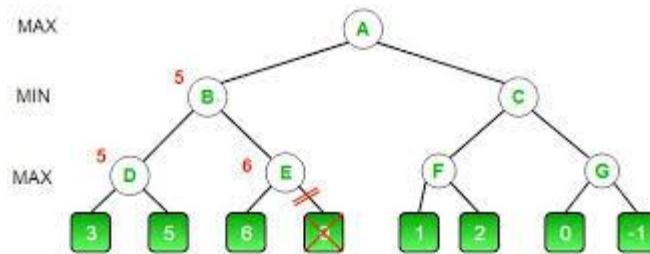
Pruning occurs:

- At a MAX node: If current value $\geq \beta$, stop exploring further children (beta cut-off)
- At a MIN node: If current value $\leq \alpha$, stop exploring further children (alpha cut-off)

These cut-offs are called pruning.

Algorithm Steps

- Start from the root node
- Pass α and β values along with each recursive call
- Update α at MAX nodes and β at MIN nodes
- Prune branches when $\alpha \geq \beta$
- Return the optimal value



2 Hill Climbing Algorithm

Local search algorithms are a class of search techniques that focus on finding an optimal solution by iteratively improving a single candidate solution. Unlike traditional search algorithms that explore a tree or graph and maintain a record of all explored paths, local search algorithms maintain only the current state in memory. These algorithms are particularly useful for optimization problems where the goal is to find the best possible state rather than the exact path to that state. Hill Climbing is one of the simplest forms of local search algorithms and is widely used for problems with large state spaces.

Hill Climbing is a greedy local search algorithm that continuously moves in the direction of increasing value (or decreasing cost).

It is analogous to: *A person climbing a hill in fog, taking steps that go uphill, hoping to reach the peak.*

Hill Climbing evaluates only the neighboring states of the current position and moves to the one that appears to be the best. If no neighbor improves the current state, the algorithm stops, potentially at a local maximum rather than the global optimum.

In hill climbing:

- A state represents a complete candidate solution
- Each state has a set of neighboring states
- Each state is assigned a value by an evaluation function

Example:

- State: position on a hill
- Neighbors: nearby positions
- Value: height of the hill

The evaluation function measures the quality of a state.

- Denoted as: $f(\text{state})$
- Higher value means better state (for maximization problems)

- Lower value means better state (for minimization problems)

Algorithm Steps

- Choose an initial state
- Loop:
 - Evaluate current state
 - Generate neighbors
 - Select the best neighbor
 - If best neighbor is better:
 - Move to that neighbor
 - Else:
 - Stop (local optimum reached)

While simple and fast, this approach has inherent limitations, such as getting stuck at local maxima, plateaus, or ridges, where the immediate neighbors do not offer improvement even though a better solution exists elsewhere.

There are several variants of Hill Climbing.

- Simple Hill Climbing examines neighbors one by one and moves to the first better neighbor found, which makes it prone to missing better alternatives.
- Steepest-Ascent Hill Climbing evaluates all neighbors and moves to the best among them, improving accuracy but requiring more computation.
- Stochastic Hill Climbing introduces randomness by selecting a random neighbor from those that improve the current state, which helps in escaping some local maxima and adds diversity to the search.

One of the main challenges of Hill Climbing is that it is not complete; it does not guarantee a solution exists for all problems. The algorithm can easily get stuck at a local maximum, a plateau, or a ridge. A local maximum occurs when a state has a higher value than all its neighbors but is not the global maximum. Plateaus are flat regions where all neighboring states have the same value, providing no direction for improvement. Ridges are narrow paths to the global maximum that cannot be reached through purely greedy moves. Due to these limitations, Hill Climbing is generally not optimal and may require enhancements to reliably reach global optima.

To address the limitations of basic Hill Climbing, several techniques can be used. **Random Restart Hill Climbing** involves running the algorithm multiple times from different initial states and selecting the best solution obtained. Sideways moves allow the algorithm to traverse plateaus by moving to neighbors with equal evaluation values. Stochastic Hill Climbing adds randomness in selecting neighbors, which helps avoid local maxima. Adjusting the neighborhood structure or increasing the neighborhood size can also improve the algorithm's ability to explore the state space effectively.

Lab Task Background

Game Playing in Artificial Intelligence

Submission Guidelines:

- Lab must be submitted in the google classroom.
- Submission other than google classroom won't not be accepted.
- You are required to submit a python (version 3 compatible) file and Notebook (Both) named after Your RollNo. e.g **LXX-XXXX.ipynb** and **LXX-XXXX.py**

Game Playing is an important domain of artificial intelligence. Games don't require much knowledge; the only knowledge we need to provide is the rules, legal moves and the conditions of winning or losing the game.

Both players try to win the game. So, both of them try to make the best move possible at each turn. Searching techniques like BFS(Breadth First Search) are not accurate for this as the branching factor is very high, so searching will take a lot of time. So, we need another search procedures that improve –

- **Generate procedure** so that only good moves are generated.
- **Test procedure** so that the best move can be explored first.

The most common search technique in game playing is **MinMax Search Algorithm**. It is depth-first depth-limited search procedure. It is used for games like **chess and tic-tac-toe**.

Minimax algorithm uses two functions –

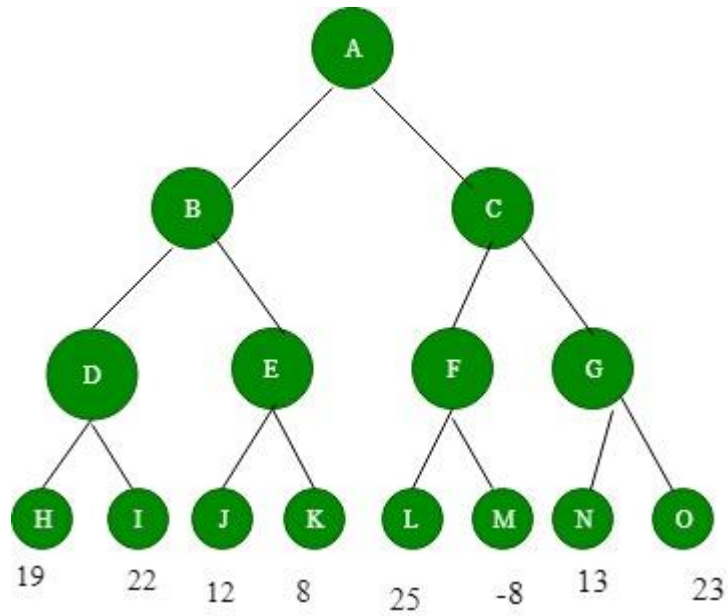
MOVEGEN : It generates all the possible moves that can be generated from the current position.

STATIC EVALUATION : It returns a value depending upon the goodness from the view point of wo-player.

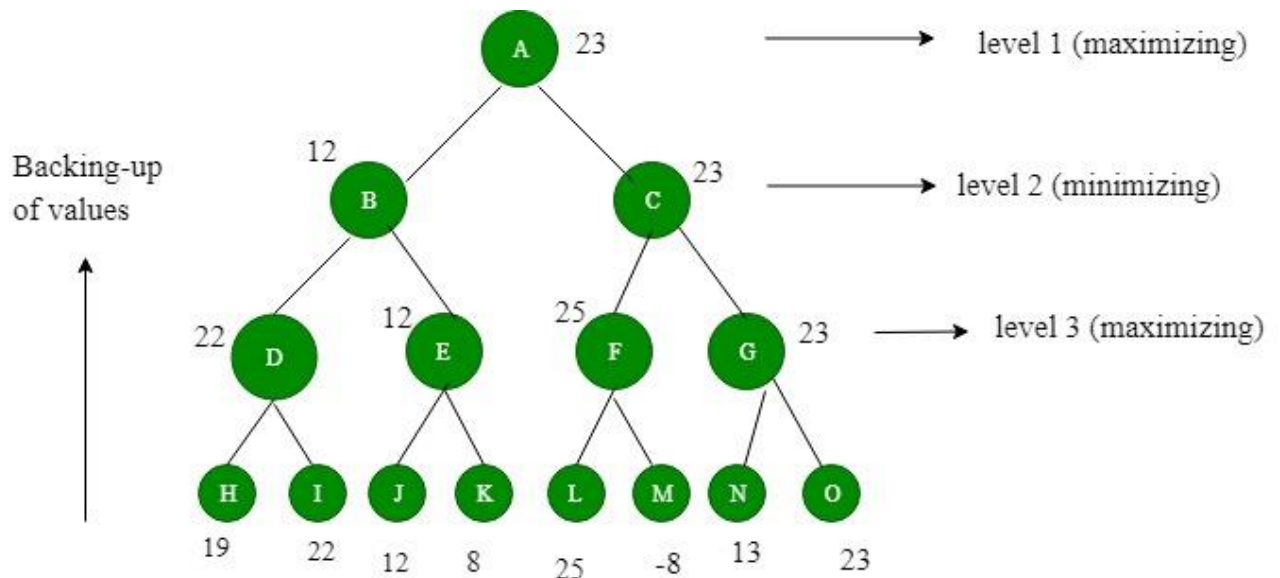
This algorithm is a two player game, so we call the first player as **PLAYER1** and second player as **PLAYER2**. The value of each node is backed-up from its children. For **PLAYER1** the backed-up value is the maximum value of its children and for **PLAYER2** the backed-up value is

the minimum value of its children. It provides most promising move to PLAYER1, assuming that the PLAYER2 has make the best move. It is a recursive algorithm, as same procedure occurs at each level.

Before Backing-up Values



After Backing-up values



Alpha-Beta Pruning

- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called **pruning**. This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prunes the tree leaves but also entire sub-tree.
- The two-parameter can be defined as:
 - Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is $-\infty$.
 - Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is $+\infty$.
- The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast

Condition for Alpha-beta pruning:

The main condition which required for alpha-beta pruning is:

$$\alpha \geq \beta$$

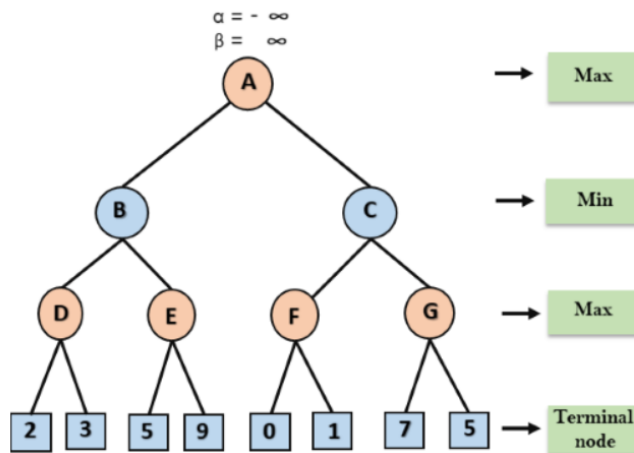
Key points about alpha-beta pruning:

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.

Working of Alpha-Beta Pruning:

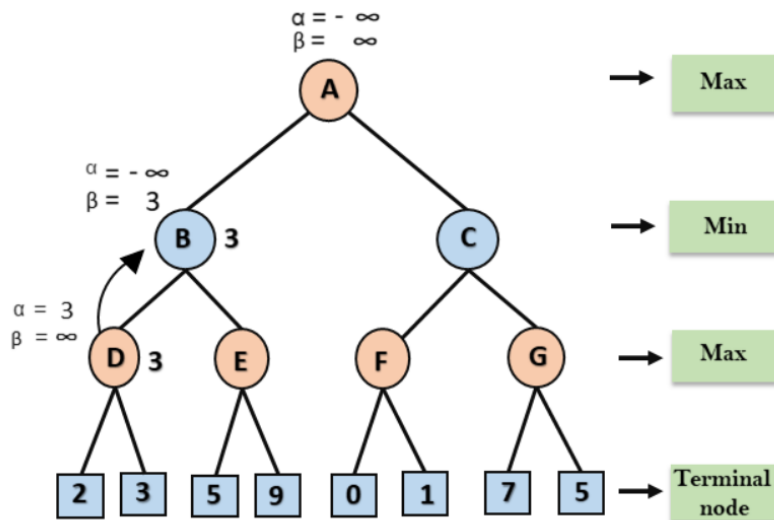
Let's take an example of two-player search tree to understand the working of Alpha-beta pruning

Step 1: At the first step the, Max player will start first move from node A where $\alpha = -\infty$ and $\beta = +\infty$, these value of alpha and beta passed down to node B where again $\alpha = -\infty$ and $\beta = +\infty$, and Node B passes the same value to its child D.



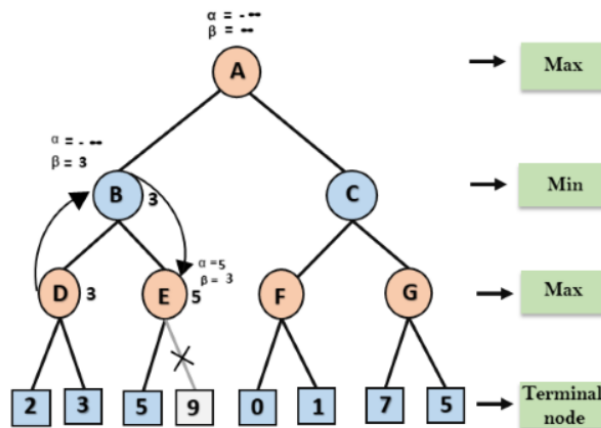
Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the $\max(2, 3) = 3$ will be the value of α at node D and node value will also 3.

Step 3: Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now $\beta = +\infty$, will compare with the available subsequent nodes value, i.e. $\min(\infty, 3) = 3$, hence at node B now $\alpha = -\infty$, and $\beta = 3$.



In the next step, algorithm traverse the next successor of Node B which is node E, and the values of $\alpha = -\infty$, and $\beta = 3$ will also be passed.

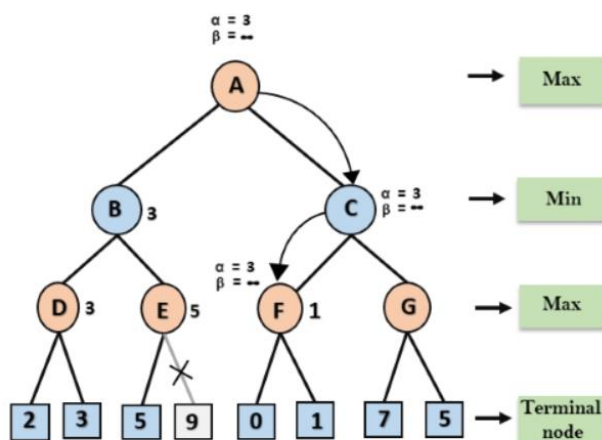
Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so $\max(-\infty, 5) = 5$, hence at node E $\alpha = 5$ and $\beta = 3$, where $\alpha > \beta$, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.



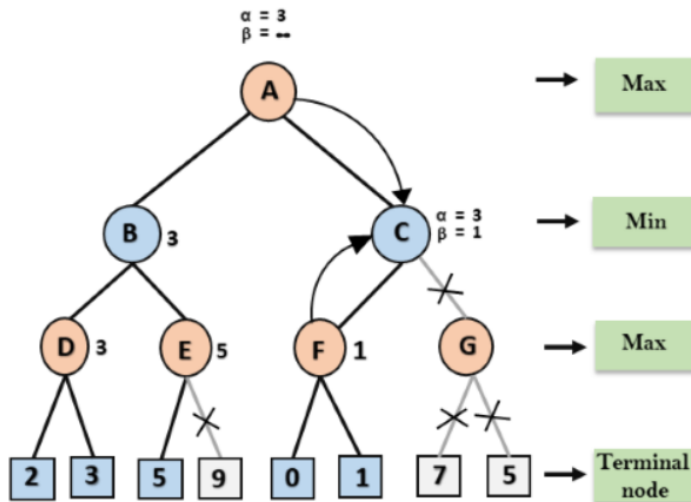
Step 5: At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as $\max(-\infty, 3) = 3$, and $\beta = +\infty$, these two values now passes to right successor of A which is Node C.

At node C, $\alpha = 3$ and $\beta = +\infty$, and the same values will be passed on to node F.

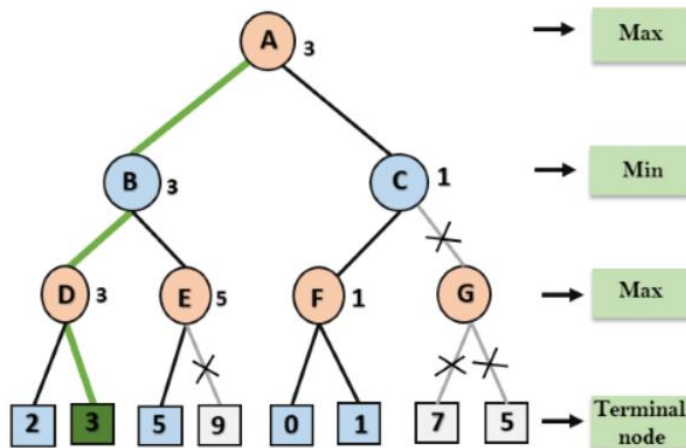
Step 6: At node F, again the value of α will be compared with left child which is 0, and $\max(3, 0) = 3$, and then compared with right child which is 1, and $\max(3, 1) = 3$ still α remains 3, but the node value of F will become 1.



Step 7: Node F returns the node value 1 to node C, at $C \alpha = 3$ and $\beta = +\infty$, here the value of beta will be changed, it will compare with 1 so $\min(\infty, 1) = 1$. Now at C, $\alpha=3$ and $\beta=1$, and again it satisfies the condition $\alpha \geq \beta$, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.



Step 8: C now returns the value of 1 to A here the best value for A is $\max(3, 1) = 3$. Following is the final game tree which is showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.

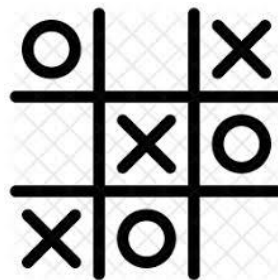


Lab Task

Artificially Intelligent TIC-TAC-TOE Game

Tic-tac-toe (also known as **noughts** and **crosses** or Xs and Os) for two players, X and O, who take turns marking the spaces in a 3×3 grid. The player who succeeds in placing three of their marks in a horizontal, vertical, or diagonal row wins the game.

The sample tic tac toe board of order 3 by 3 is shown in the figure below. Suppose player 1 had chosen X and player 2 has chosen O.



Problem Statement

You need to implement **Min-Max Algorithm** to search optimal solution for each player in Tic-Tac-Toe game. (Algorithm is given at the end for your reference)

Rules of Game :

The simple rules to play the game are:

- Two players can play the game. Each player will choose either O or X.
- The winning of a player is dependent on the consecutive Os or Xs (in either row or column or Diagonal).
- Each player will play alternately. There is no biasness towards or against any player.
- The game will stop when either player has won.
- If no player has won and all the board cells are occupied, the game has been drawn.

Code Provided

def initial_state(): Returns starting state of the board.
def player(board): Returns player who has the next turn on a board.
def actions(board): Returns set of all possible actions (i, j) available on the board.
def result(board, action): Returns the board that results from making move (i, j) on the board.
def winner(board): Returns the winner of the game, if there is one.
def terminal(board): Returns True if game is over, False otherwise.
def utility(board): Returns 1 if X has won the game, -1 if O has won, 0 otherwise.

Task to Perform

- 1- **def alpha_beta_pruning(board):** Returns the optimal action for the current player on the board.
- 2- Task 2: Solve Travelling Salesman Problem using Hill Climbing Algorithm.

TSM problem description:

The Traveling Salesman Problem (TSP)

Definition: Given a list of cities and the distances between each pair, the goal is to find the Hamiltonian cycle with the minimum total weight (distance or cost).

Complexity: The number of possible routes grows factorially with the number of cities, making exact (brute-force) solutions computationally infeasible for large instances.

Applications: TSP has practical applications in logistics, vehicle routing, manufacturing, and tourism planning.

International Journal for Simulation and Multidisciplinary Design Optimization

International Journal for Simulation and Multidisciplinary Design Optimization

Hill Climbing Algorithm:

Hill climbing is a greedy, iterative local search algorithm that starts with an initial solution and repeatedly moves to a better neighboring solution until no further improvement can be found.

How Hill Climbing is Applied to TSP:

Generate an initial solution: A random sequence of cities is created as the starting route.

Define "neighbors": Neighboring solutions are typically generated by making small changes, such as swapping the positions of two cities in the current route.

Evaluate neighbors: The total distance (objective function) of the current route and its neighbors is calculated.

Move to a better solution: The algorithm selects the neighbor with the shortest distance (the "steepest ascent" or "simple" variant) and makes it the new current solution.

Repeat: This process continues until no neighbor is better than the current solution, at which point the algorithm terminates, having found a local optimum.

Relationship and Limitations

Heuristic Approach: Hill climbing is a heuristic, meaning it provides a good, but not necessarily perfect, solution in a reasonable amount of time.

Local Optima: The main weakness is that it can get trapped in a local optimum—a solution better than its immediate neighbors, but potentially far from the global optimum (the true shortest path).

Hybrid Solutions: To overcome the local optima problem, hill climbing is often combined with other techniques like [simulated annealing](<https://www.geeksforgeeks.org/artificial-intelligence/hill-climbing-and-simulated-annealing-for-the-traveling-salesman-problem/>) or genetic algorithms, which allow occasional moves to worse solutions to explore the search space more thoroughly.

In summary, hill climbing is a simple and fast method for tackling the complex TSP, especially useful when an exact optimal solution is not strictly required, and a "good enough" solution is acceptable within time constraints.